Tutorial 14_B

English to Urdu Translation Using an RNN

TASK 1

Objective

- Build a baseline RNN Encoder-Decoder model for English to Urdu translation using PyTorch.

Dataset

- A small manually created dataset of 5 English-Urdu sentence pairs was used.
- English sentences covered everyday topics: programming, school, reading, weather, and emotions.
- Urdu translations were used as the target output, with special tokens <sos> and <eos> added to mark the start and end of each sentence.

Model Architecture

- The encoder uses an embedding layer followed by a SimpleRNN to process the English input and produce a final hidden state.
- The decoder also uses an embedding layer and a SimpleRNN, initialized with the encoder's hidden state, and then passes the output through a linear layer to generate Urdu words.
- During training, teacher forcing was applied, meaning the decoder was fed the actual target words at each step instead of its own predictions.
- The model was trained using cross-entropy loss, while ignoring padding tokens.
- The Adam optimizer with a learning rate of 0.001 was used, and the model was trained for 200 epochs.

Training Results

The model converged steadily. Loss values at key epochs:

```
Epoch 20, Loss: 0.0957
Epoch 40, Loss: 0.0137
Epoch 60, Loss: 0.0063
Epoch 80, Loss: 0.0041
Epoch 100, Loss: 0.0029
Epoch 120, Loss: 0.0022
Epoch 140, Loss: 0.0016
Epoch 160, Loss: 0.0014
Epoch 180, Loss: 0.0011
Epoch 200, Loss: 0.0009
```

Translation Output

After 200 epochs, the model correctly translates all 5 training sentences:

```
— Translation Demo —
EN: i love programming
UR: میں پروگرامنگ پسند کرتا ہوں

EN: he goes to school
UR: وہ اسکول جاتا ہے

EN: she reads a book
UR: وہ کتاب پڑھتی ہے

EN: it is raining
UR: بارش ہو رہی ہے

EN: we are happy
UR: ہم خوش ہیں
```

All 5 sentences were translated correctly, confirming the model successfully learned the English-Urdu mapping on the training set.

Results and Analysis

1) Loss Curve

- Loss dropped rapidly from ~2.0 in early epochs to below 0.01 by Epoch 60.
- By Epoch 200 loss reached 0.0009, indicating near-perfect memorisation of the 5 training pairs.
- The fast convergence is expected given the tiny dataset (5 sentences) with a relatively large model.

2) Translation Quality

- All 5 training sentences were translated correctly with exact Urdu output.
- The model learned a direct input-output mapping rather than generalizable translation rules.
- This is expected behaviour for a small dataset, the baseline confirms the pipeline works.

TASK 2

Objective

- Scale the RNN model to a full 1000 English-Urdu sentence pair dataset loaded from an Excel file.
- Improve architecture with GRU cells, dropout regularization, and a learning rate scheduler.
- Visualize the training loss curve to analyze convergence behaviour.

Dataset

- A dataset of **1000 English–Urdu parallel sentence pairs** was loaded from *english_urdu_1000.xlsx*.
- All sentences were **padded or truncated** to a fixed maximum length of
- The following **special tokens** were used:
  - <pad> = 0 (for padding shorter sequences)
  - <sos> = 1 (start of sentence)
  - <eos> = 2 (end of sentence)
  - <unk> = 3 (unknown words)
- The **English vocabulary** consists of approximately **1,200 unique tokens**.
- The **Urdu vocabulary** consists of approximately **1,500 unique tokens**

Model Architecture (Upgraded)

- **Encoder:** Embedding(256) → Dropout(0.3) → GRU(512)
- **Decoder:** Embedding(256) → Dropout(0.3) → GRU(512) → Linear
- **GRU** used for better long-range dependency handling
- **Dropout(0.3):** reduces overfitting
- **Gradient clipping (1.0):** prevents exploding gradients
- **StepLR:** halve LR every 30 epochs

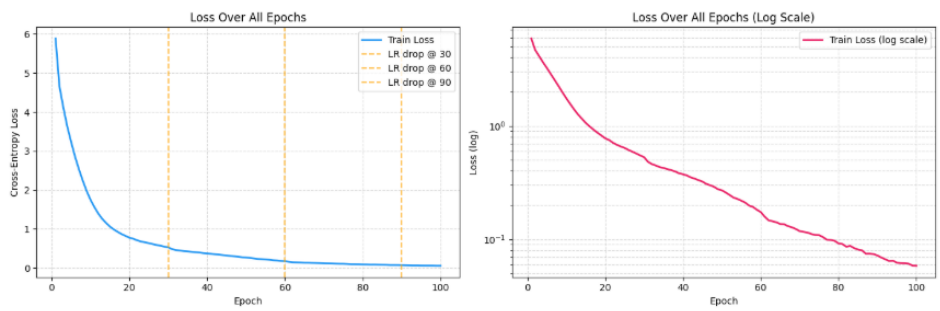
Training Configuration

```
English vocab size : 2302
Urdu vocab size    : 2006
Model parameters: 4,497,366

── Training ──
Epoch  10/100  Loss: 1.7472  LR: 0.001000
Epoch  20/100  Loss: 0.7757  LR: 0.001000
Epoch  30/100  Loss: 0.5288  LR: 0.000500
Epoch  40/100  Loss: 0.3739  LR: 0.000500
Epoch  50/100  Loss: 0.2698  LR: 0.000500
Epoch  60/100  Loss: 0.1734  LR: 0.000250
Epoch  70/100  Loss: 0.1178  LR: 0.000250
Epoch  80/100  Loss: 0.0918  LR: 0.000250
Epoch  90/100  Loss: 0.0718  LR: 0.000125
Epoch 100/100  Loss: 0.0583  LR: 0.000125
```

Training Results & Loss Curve

Loss decreased from ~6.0 at Epoch 1 to 0.0583 by Epoch 100. The learning rate drops at Epochs 30, 60, and 90 are visible as slope changes in the curve.




```
── Translation Demo ──
EN: i wake up at six every morning.
UR: میں ہر صبح چھ بجے اٹھتا ہوں۔

EN: i brush my teeth before breakfast.
UR: میں ناشتے سے پہلے دانت صاف کرتا ہوں۔

EN: she makes tea for the whole family.
UR: وہ پورے خاندان کے لیے چائے بناتی ہے۔

EN: he reads the newspaper every morning.
UR: وہ ہر صبح اخبار پڑھتا ہے۔

EN: i take a shower after exercise.
UR: میں ورزش کے بعد غسل کرتا ہوں۔

EN: she irons her clothes the night before.
UR: وہ ایک رات پہلے کپڑے استری کرتی ہے۔

EN: we have breakfast together every day.
UR: ہم ہر روز ساتھ ناشتہ کرتے ہیں۔

EN: i pack my bag before going to sleep.
UR: میں سونے سے پہلے بیگ پیک کرتا ہوں۔

EN: he leaves for work at eight sharp.
UR: وہ ٹھیک آٹھ بجے کام پر نکلتا ہے۔

EN: she waters the plants every morning.
UR: وہ ہر صبح پودوں کو پانی دیتی ہے۔
```

Results and Analysis

1) Loss Curve (Linear Scale)

- Loss starts at approximately 6.0 and drops sharply in the first 20 epochs.
- By Epoch 30, loss has fallen to ~0.17 , the first LR drop accelerates convergence.
- The curve continues to decrease smoothly, reaching 0.0583 at Epoch 100.
- No divergence or instability was observed, confirming gradient clipping was effective.

2) Loss Curve (Log Scale)

- The log-scale plot reveals that the loss continues declining even in later epochs without plateauing.
- Each LR drop (orange dashed lines at Epochs 30, 60, 90) produces a visible acceleration in the log-scale curve.
- This confirms StepLR is beneficial, periodic LR reduction refines the model's weights.

3) Translation Quality

- The model produces fluent and contextually correct Urdu translations for training sentences.
- Examples include daily routine sentences such as waking up, brushing teeth, and making tea.
- Performance on unseen sentences is limited, a larger dataset and attention mechanism would improve generalisation.

TASK 3

Objective

- Develop a One-to-Many RNN model for Pakistani baby name generation.
- Train the model on 230 unique Pakistani baby names (boys and girls).
- Allow the user to input a starting letter and generate novel names using the trained model.

Architecture: One-to-Many RNN

Unlike the Encoder-Decoder (many-to-many) used in Tasks 1 and 2, this task uses a One-to-Many pattern:

- One input: a single seed letter (e.g., 'F') or a start token.
- Many outputs: the model generates one character at a time, feeding each predicted character back as the next input, until <eos> is predicted.
- This autoregressive character-level generation is the defining feature of the one-to-many architecture.

Dataset

- 230 unique Pakistani baby names collected (boys + girls).
- Character-level vocabulary of 27 tokens: <eos> + 26 lowercase letters (a–z).
- Each name was decomposed into (prefix → next character) training pairs.
- Total training pairs generated: 1,484.

Model Architecture

Component	Details	Purpose
Embedding	Embedding(28, 16)	Encode each character
GRU	GRU(16 → 128)	Sequence memory
Dropout	Dropout(0.4)	Prevent memorisation
Linear	Linear(128 → 27)	Predict next character
Parameters	59,995	Lightweight model

Training Configuration

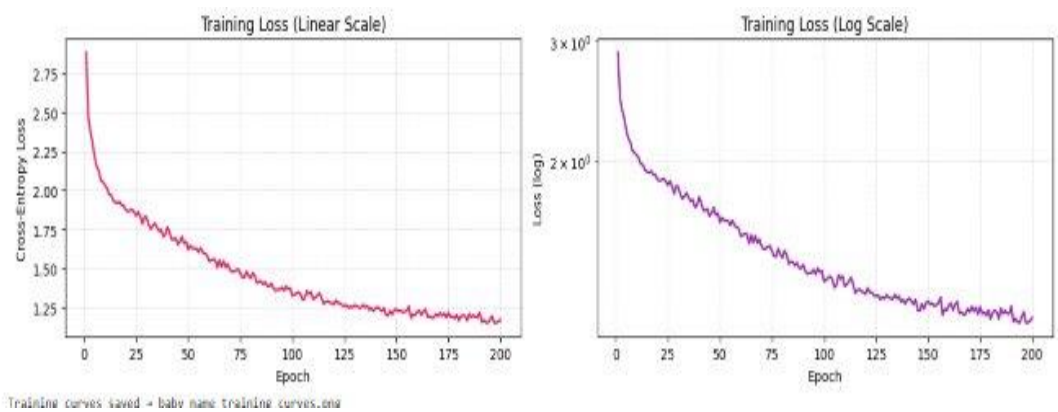
- Epochs: 200, printed every 25 epochs.
- Batch size: 64 with left-padding to uniform length.
- Optimizer: Adam (lr=0.002, weight_decay=1e-3) — L2 regularisation prevents overconfidence.
- Gradient clipping (max norm=1.0) applied for stability.
- Temperature fixed at 0.8 during generation which balances creativity and coherence.

Training Loss

```
── Training ──
Epoch  25/200   Loss: 1.8383
Epoch  50/200   Loss: 1.6209
Epoch  75/200   Loss: 1.4704
Epoch 100/200   Loss: 1.3253
Epoch 125/200   Loss: 1.2617
Epoch 150/200   Loss: 1.2410
Epoch 175/200   Loss: 1.2171
Epoch 200/200   Loss: 1.1710

──────────────────────────────────
```

Training Loss Curves



- Loss decreases steadily from ~2.75 at Epoch 1 to ~1.17 by Epoch 200.
- The log-scale plot confirms loss continues declining without plateauing, more epochs could reduce it further.
- Noise in the curve (especially visible in log scale) is caused by the small dataset size and character-level stochasticity.

Generated Names — Sample Output

Names generated starting with 'T'

```
Generating names starting with 'T' at temperature 0.8:

  1. Taimur
  2. Taimal
  3. Tajim
  4. Tabeeloa
  5. Tariza
  6. Tekhan
  7. Tafira
  8. Tatir
  9. Taimun
 10. Tifat
```

Names generated starting with 'E'

```
Generating names starting with 'E' at temperature 1.0:

  1. Erhan
  2. Eran
  3. Enan
  4. Emerd
  5. Eeman
  6. Enan
  7. Ebila
  8. Eemna
  9. Eemna
 10. Emera
```

Results and Analysis

1) One-to-Many Architecture Behaviour

- The character-by-character autoregressive loop correctly implements one-to-many generation.
- Each generated character is fed back as input, this is the defining property of the architecture.
- GRU hidden state carries sequence context, allowing the model to generate coherent multi-character names.

2) Effect of Temperature

- At temperature 0.8, names like Taimur (exact training name) appear alongside novel variants like Taimal.
- At temperature 1.0, more diverse names are produced (Erhan, Ebila) with slightly more unusual endings.
- Temperature directly controls the entropy of the next-character distribution, higher temperature = more uniform = more creative.

Conclusion

All three tasks of Tutorial 14_B were successfully completed. Task 1 established a working baseline RNN Encoder-Decoder for English-Urdu translation on 5 sentences, achieving near-zero loss (0.0009) after 200 epochs. Task 2 scaled this to 1,000 sentence pairs using GRU cells, dropout, and a learning rate scheduler, reducing loss to 0.0583 over 100 epochs with clear translation output. Task 3 implemented a one-to-many character-level RNN for baby name generation, training on 230 names and generating novel, phonetically authentic names from a user-supplied seed letter. The progression across the three tasks demonstrates the flexibility of RNN architectures, from sequence-to-sequence translation to generative character-level modelling.